

UNIT 1 – 2 MARKS QUESTIONS (Short Answers)

1. What is JDBC?

JDBC (Java Database Connectivity) is an API used to connect Java applications with relational databases. It allows executing SQL queries and retrieving results.

2. Define DriverManager.

DriverManager is a class that manages all database drivers and establishes a connection between Java application and database.

3. What is a JDBC Driver?

A JDBC Driver is a software component that enables Java programs to interact with a database.

4. What is ResultSet?

ResultSet is an object that stores the data returned after executing a SELECT SQL query.

5. What is metadata in JDBC?

Metadata provides information about database, tables, columns and SQL types using DatabaseMetaData and ResultSetMetaData.

6. Define PreparedStatement.

PreparedStatement is a precompiled SQL statement used for executing dynamic queries securely and faster.

7. Define CallableStatement.

CallableStatement is used to call stored procedures in the database.

8. Define POM.xml in Maven.

POM.xml (Project Object Model) is the main configuration file in Maven that contains project details, dependencies, plugins and build settings.

9. What is Maven Repository?

It is a storage location where Maven stores libraries, dependencies, and plugins.

10. Define JDBC URL.

A JDBC URL is a connection string used by Java to identify the database location.

✔ UNIT 1 – 4 MARKS QUESTIONS (Moderate Answers)

1. Explain JDBC architecture with neat diagram.

JDBC architecture consists of two layers:

1. **JDBC API** – Interfaces like Connection, Statement, ResultSet.
2. **JDBC Driver Manager** – Selects correct driver.

JDBC uses four components:

- Application
- JDBC Driver Manager
- JDBC Driver
- Database

The application sends SQL requests → DriverManager selects driver → Driver communicates with DB → Results return to application.

2. Explain Statement vs PreparedStatement.

Statement	PreparedStatement
Used for static SQL queries.	Used for dynamic/precompiled SQL.
No parameter support.	Supports parameters using ?.
Slow performance.	Faster due to precompilation.
Vulnerable to SQL injection.	Secure and safe.

3. Explain different types of JDBC Drivers.

There are **4 types of JDBC Drivers**:

1. **Type-1: JDBC-ODBC Bridge Driver** – Uses ODBC, slow.
 2. **Type-2: Native API Driver** – Uses DB client libraries.
 3. **Type-3: Network Protocol Driver** – Uses middleware server.
 4. **Type-4: Thin Driver** – Pure Java driver, best performance, platform-independent.
-

4. Explain Maven lifecycle phases.

Maven has 3 main lifecycles:

1. **Clean Lifecycle** – Removes previous build (clean).

2. **Default Lifecycle** – Compiles, tests, packages, installs the project.
 3. **Site Lifecycle** – Generates project documentation.
Inside Default lifecycle: validate → compile → test → package → verify → install → deploy.
-

5. Explain MySQL connection steps in JDBC.

1. **Load Driver:** `Class.forName("com.mysql.cj.jdbc.Driver")`
 2. **Create Connection:** Using `DriverManager.getConnection()`.
 3. **Create Statement:** Using `createStatement()` or `PreparedStatement`.
 4. **Execute Query:** `executeQuery()` or `executeUpdate()`.
 5. **Close Connection:** Close `ResultSet`, `Statement`, `Connection`.
-

✓ UNIT 1 – 5 MARKS QUESTIONS (Detailed Answers)

1. Explain steps of JDBC connectivity with code.

JDBC connectivity steps:

1. **Import packages** – `java.sql.*`
2. **Load driver**
3. **Establish connection**
4. **Create statement**
5. **Execute query**
6. **Process the results**
7. **Close the connection**

Code:

```
Class.forName("com.mysql.cj.jdbc.Driver");  
  
Connection con = DriverManager.getConnection(  
"jdbc:mysql://localhost:3306/test","root","password");  
  
Statement st = con.createStatement();  
  
ResultSet rs = st.executeQuery("SELECT * FROM student");
```

```
while(rs.next()){  
    System.out.println(rs.getString(1));  
}  
con.close();
```

2. Explain Working of DriverManager.

- Acts as a mediator between application and drivers.
 - Maintains a list of registered drivers.
 - When getConnection() is called, DriverManager:
 1. Searches for suitable driver
 2. Passes the connection URL to driver
 3. Opens connection and returns it to the application
 - Ensures correct driver selection and manages multiple drivers efficiently.
-

3. Describe all JDBC Drivers in detail.

Type 1 – JDBC-ODBC Bridge:

- Converts JDBC calls into ODBC calls.
- Requires ODBC installation.
- Slow and platform-dependent.

Type 2 – Native API Driver:

- Uses vendor-specific DB libraries.
- Faster than Type-1 but not portable.

Type 3 – Network Protocol Driver:

- Uses middleware server to translate JDBC calls.
- Good for enterprise apps.

Type 4 – Thin Driver:

- Pure Java driver.
- Directly communicates with database.

- Best performance and platform-independent.
-

4. Explain Maven project directory structure.

Typical structure:

project/

src/

main/

java/ → Application source files

resources/ → Config files

test/

java/ → Test cases

target/ → Output files

pom.xml → Project configuration

- **pom.xml** contains project dependencies.
 - **src/main/java** contains code.
 - **src/main/resources** contains XML, properties.
 - **target** contains compiled .class files, JARs.
-

✓ UNIT 1 – 6 MARKS QUESTIONS (Half-Page Answers)

Q1A (a + b + c)

(a) What is JDBC?

JDBC is a Java API that helps Java applications connect and interact with relational databases. It provides interfaces such as Connection, Statement, and ResultSet for executing SQL operations.

(b) Explain types of JDBC drivers.

Four JDBC driver types:

1. JDBC-ODBC Bridge

2. Native API driver
3. Network Protocol driver
4. Thin driver

Each type differs in performance, portability and implementation.

(c) Explain steps of JDBC connection.

1. Load driver
 2. Establish connection
 3. Create Statement
 4. Execute SQL commands
 5. Process ResultSet
 6. Close connection
-

Q1B (a + b + c)

(a) What is POM.xml?

POM.xml is the core configuration file in Maven that defines dependencies, plugins, repositories, versioning, and build instructions.

(b) Explain Maven lifecycle.

Three lifecycles: Clean, Default, Site.

Default lifecycle includes: validate → compile → test → package → install → deploy.

(c) Explain dependency management.

Maven downloads and manages project libraries from repositories automatically. Developers only specify version and artifact in POM.xml, reducing manual jar management.

UNIT 1 – 2 MARKS QUESTIONS (Short Answers)

1. What is JDBC?

JDBC (Java Database Connectivity) is an API used to connect Java applications with relational databases. It allows executing SQL queries and retrieving results.

2. Define DriverManager.

DriverManager is a class that manages all database drivers and establishes a connection between Java application and database.

3. What is a JDBC Driver?

A JDBC Driver is a software component that enables Java programs to interact with a database.

4. What is ResultSet?

ResultSet is an object that stores the data returned after executing a SELECT SQL query.

5. What is metadata in JDBC?

Metadata provides information about database, tables, columns and SQL types using DatabaseMetaData and ResultSetMetaData.

6. Define PreparedStatement.

PreparedStatement is a precompiled SQL statement used for executing dynamic queries securely and faster.

7. Define CallableStatement.

CallableStatement is used to call stored procedures in the database.

8. Define POM.xml in Maven.

POM.xml (Project Object Model) is the main configuration file in Maven that contains project details, dependencies, plugins and build settings.

9. What is Maven Repository?

It is a storage location where Maven stores libraries, dependencies, and plugins.

10. Define JDBC URL.

A JDBC URL is a connection string used by Java to identify the database location.

✔ UNIT 1 – 4 MARKS QUESTIONS (Moderate Answers)

1. Explain JDBC architecture with neat diagram.

JDBC architecture consists of two layers:

1. **JDBC API** – Interfaces like Connection, Statement, ResultSet.
2. **JDBC Driver Manager** – Selects correct driver.

JDBC uses four components:

- Application
- JDBC Driver Manager
- JDBC Driver
- Database

The application sends SQL requests → DriverManager selects driver → Driver communicates with DB → Results return to application.

2. Explain Statement vs PreparedStatement.

Statement

Used for static SQL queries.

No parameter support.

Slow performance.

Vulnerable to SQL injection.

PreparedStatement

Used for dynamic/precompiled SQL.

Supports parameters using ?.

Faster due to precompilation.

Secure and safe.

3. Explain different types of JDBC Drivers.

There are **4 types of JDBC Drivers**:

1. **Type-1: JDBC-ODBC Bridge Driver** – Uses ODBC, slow.
 2. **Type-2: Native API Driver** – Uses DB client libraries.
 3. **Type-3: Network Protocol Driver** – Uses middleware server.
 4. **Type-4: Thin Driver** – Pure Java driver, best performance, platform-independent.
-

4. Explain Maven lifecycle phases.

Maven has 3 main lifecycles:

1. **Clean Lifecycle** – Removes previous build (clean).

2. **Default Lifecycle** – Compiles, tests, packages, installs the project.
 3. **Site Lifecycle** – Generates project documentation.
Inside Default lifecycle: validate → compile → test → package → verify → install → deploy.
-

5. Explain MySQL connection steps in JDBC.

1. **Load Driver:** `Class.forName("com.mysql.cj.jdbc.Driver")`
 2. **Create Connection:** Using `DriverManager.getConnection()`.
 3. **Create Statement:** Using `createStatement()` or `PreparedStatement`.
 4. **Execute Query:** `executeQuery()` or `executeUpdate()`.
 5. **Close Connection:** Close `ResultSet`, `Statement`, `Connection`.
-

✓ UNIT 1 – 5 MARKS QUESTIONS (Detailed Answers)

1. Explain steps of JDBC connectivity with code.

JDBC connectivity steps:

1. **Import packages** – `java.sql.*`
2. **Load driver**
3. **Establish connection**
4. **Create statement**
5. **Execute query**
6. **Process the results**
7. **Close the connection**

Code:

```
Class.forName("com.mysql.cj.jdbc.Driver");  
  
Connection con = DriverManager.getConnection(  
"jdbc:mysql://localhost:3306/test","root","password");  
  
Statement st = con.createStatement();  
  
ResultSet rs = st.executeQuery("SELECT * FROM student");
```

```
while(rs.next()){  
    System.out.println(rs.getString(1));  
}  
con.close();
```

2. Explain Working of DriverManager.

- Acts as a mediator between application and drivers.
 - Maintains a list of registered drivers.
 - When getConnection() is called, DriverManager:
 1. Searches for suitable driver
 2. Passes the connection URL to driver
 3. Opens connection and returns it to the application
 - Ensures correct driver selection and manages multiple drivers efficiently.
-

3. Describe all JDBC Drivers in detail.

Type 1 – JDBC-ODBC Bridge:

- Converts JDBC calls into ODBC calls.
- Requires ODBC installation.
- Slow and platform-dependent.

Type 2 – Native API Driver:

- Uses vendor-specific DB libraries.
- Faster than Type-1 but not portable.

Type 3 – Network Protocol Driver:

- Uses middleware server to translate JDBC calls.
- Good for enterprise apps.

Type 4 – Thin Driver:

- Pure Java driver.
- Directly communicates with database.

- Best performance and platform-independent.
-

4. Explain Maven project directory structure.

Typical structure:

project/

src/

main/

java/ → Application source files

resources/ → Config files

test/

java/ → Test cases

target/ → Output files

pom.xml → Project configuration

- **pom.xml** contains project dependencies.
 - **src/main/java** contains code.
 - **src/main/resources** contains XML, properties.
 - **target** contains compiled .class files, JARs.
-

✓ UNIT 1 – 6 MARKS QUESTIONS (Half-Page Answers)

Q1A (a + b + c)

(a) What is JDBC?

JDBC is a Java API that helps Java applications connect and interact with relational databases. It provides interfaces such as Connection, Statement, and ResultSet for executing SQL operations.

(b) Explain types of JDBC drivers.

Four JDBC driver types:

1. JDBC-ODBC Bridge

2. Native API driver
3. Network Protocol driver
4. Thin driver

Each type differs in performance, portability and implementation.

(c) Explain steps of JDBC connection.

1. Load driver
 2. Establish connection
 3. Create Statement
 4. Execute SQL commands
 5. Process ResultSet
 6. Close connection
-

Q1B (a + b + c)

(a) What is POM.xml?

POM.xml is the core configuration file in Maven that defines dependencies, plugins, repositories, versioning, and build instructions.

(b) Explain Maven lifecycle.

Three lifecycles: Clean, Default, Site.

Default lifecycle includes: validate → compile → test → package → install → deploy.

(c) Explain dependency management.

Maven downloads and manages project libraries from repositories automatically. Developers only specify version and artifact in POM.xml, reducing manual jar management.

UNIT 3 – 2 MARKS QUESTIONS

1. What is JSP?

JSP (Java Server Pages) is a server-side technology that allows writing dynamic web pages using HTML and Java code embedded inside special tags.

2. Define scripting element.

Scripting elements allow embedding Java code inside a JSP page. Examples: Scriptlet, Declaration, Expression.

3. What is a scriptlet?

A scriptlet is Java code written inside `<% ... %>` tags and executed during request processing.

4. What is declaration?

A declaration is written inside `<%! ... %>` and is used to declare variables or methods that become part of the servlet class.

5. Define expression tag.

Expression tag `<%= ... %>` is used to output values directly to the web page.

6. What is JSP action tag?

Action tags are XML-based tags used to control the behavior of JSP, e.g., `<jsp:include>`, `<jsp:forward>`, `<jsp:useBean>`.

7. What is include directive?

Include directive `<%@ include file="..." %>` inserts content from another file during JSP translation time.

8. What are implicit objects?

Implicit objects are predefined objects available in JSP such as request, response, session, application, config, out, pageContext.

UNIT 3 – 4 MARKS QUESTIONS

1. Explain JSP lifecycle.

The JSP lifecycle includes:

1. **Translation** – JSP is converted into a servlet.
2. **Compilation** – Converted servlet is compiled to bytecode.

3. **Class Loading** – Class loader loads the servlet.
 4. **Instantiation** – Servlet instance is created.
 5. **Initialization (jspInit())** – Called once.
 6. **Request Processing (_jspService())** – Called for each request.
 7. **Destruction (jspDestroy())** – Called before removal from memory.
-

2. Explain scripting elements.

Three scripting elements:

1. **Scriptlet (<% %>)** – Contains Java code to handle logic.
 2. **Expression (<%= %>)** – Prints values directly on page.
 3. **Declaration (<%! %>)** – Defines variables/methods at class level.
- These elements help in writing dynamic content inside HTML.
-

3. Explain directive elements.

Directive elements provide global instructions to JSP container.

Three types:

1. **Page Directive** – Sets page-level properties (language, import, session).
 2. **Include Directive** – Includes file at translation time.
 3. **Taglib Directive** – Declares custom tag libraries.
-

4. Explain implicit objects in JSP.

Implicit objects are built-in objects provided by JSP container:

- **request** – Handles request data.
- **response** – Sends output.
- **session** – Stores user session.
- **application** – Application-wide data.
- **config** – Servlet configuration.
- **out** – Writes output.

- **pageContext** – Provides access to all JSP features.
They simplify request handling and output generation.
-

✓ UNIT 3 – 5 MARKS QUESTIONS

1. Compare JSP and Servlet.

JSP:

- Used mainly for presentation layer.
- Easier to write because of HTML support.
- Automatically converted into servlet.
- Uses implicit objects.

Servlet:

- Used for backend processing.
- Harder to write HTML code.
- No automatic conversion.
- Needs manual handling of output.

Conclusion: JSP is best for UI; Servlet is best for business logic.

2. Explain CRUD operations using JSP.

CRUD refers to **Create, Read, Update, Delete**:

1. **Create:** JSP form submits data to INSERT SQL query using Servlet/JDBC.
 2. **Read:** JSP fetches and displays records in HTML table.
 3. **Update:** Forms allow editing of existing data → UPDATE SQL.
 4. **Delete:** Deletes record based on ID → DELETE SQL.
- JSP is used for UI, while Servlets handle business logic.
-

3. Explain JSP directives.

Directives control how JSP page is processed.

1. Page Directive

Used to define attributes like language, buffer, session, import:

Example:

```
<%@ page import="java.sql.*" %>
```

2. Include Directive

Includes a file during translation:

```
<%@ include file="header.jsp" %>
```

3. Taglib Directive

Defines custom tag library:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

These help in modular and maintainable JSP development.

UNIT 3 – 6 MARKS QUESTION (2+2+2)

(a) Explain scripting elements.

There are three scripting elements:

- **Scriptlet:** Allows writing Java code inside `<% %>`.
- **Expression:** Prints output using `<%= %>`.
- **Declaration:** Defines methods and variables using `<%! %>`.
These are used for logic, output, and class-level declarations.

(b) Explain page directive.

Page directive gives information to JSP engine about the page.

Syntax:

```
<%@ page attribute="value" %>
```

Important attributes:

- `import` – imports Java packages
- `session` – enables session
- `buffer` – size of output buffer
- `contentType` – response type
It controls behavior of the entire JSP page.

(c) Explain JSP lifecycle.

JSP lifecycle includes steps:

1. Translation → JSP to servlet
2. Compilation → servlet to class
3. Loading → class loaded
4. Instantiation → object created
5. `jspInit()` → initialization
6. `_jspService()` → handles requests
7. `jspDestroy()` → cleanup

UNIT 4 – 2 MARKS QUESTIONS

1. What is ORM?

ORM (Object Relational Mapping) is a technique that maps Java objects to database tables, allowing developers to interact with databases using objects instead of SQL.

2. Define Hibernate.

Hibernate is an ORM framework for Java that simplifies database operations by mapping Java classes to database tables.

3. What is Configuration?

Configuration is a class in Hibernate used to load Hibernate properties, configurations, and mapping files.

4. What is SessionFactory?

SessionFactory is a heavyweight object used to create Session objects. It is created once per application.

5. What is Session?

A Session represents a single unit of work with the database, used to perform CRUD operations.

6. What is Transaction?

A Transaction ensures that a set of operations execute as a single unit. It provides commit and rollback functionality.

7. What is HQL?

HQL (Hibernate Query Language) is an object-oriented query language similar to SQL but operates on objects instead of tables.

8. What is Criteria API?

Criteria API is a dynamic query generation mechanism in Hibernate used to build queries programmatically.

UNIT 4 – 4 MARKS QUESTIONS

1. Explain Hibernate architecture.

Hibernate architecture consists of the following components:

1. **Configuration** – Loads configuration settings and mapping.
2. **SessionFactory** – Creates Sessions; heavy object.
3. **Session** – Represents a connection with the database and performs CRUD.
4. **Transaction** – Manages commit and rollback.
5. **Query / HQL** – Used for retrieving and manipulating data.
6. **JDBC** – Hibernate uses JDBC internally for communication.

Together, these components form a powerful abstraction for database operations.

2. Explain lifecycle of a Hibernate Session.

Session lifecycle has three stages:

1. **Open Session** – Created by SessionFactory.
2. **Perform Operations** – CRUD, HQL queries, transactions.
3. **Close Session** – Frees database connection and resources.

Session persists objects in the first-level cache until closed.

3. Explain Hibernate mapping.

Mapping defines how Java classes and properties map to database tables and columns.

Types of mapping:

- **Class-to-table mapping**
- **Property-to-column mapping**
- **Association mapping (One-to-One, One-to-Many, Many-to-Many)**

Mapping can be done in:

- **XML (.hbm.xml)**
 - **Annotations (@Entity, @Table, @Column)**
-

4. Explain Hibernate annotations.

Hibernate supports JPA-style annotations:

- **@Entity** – Marks class as table.
- **@Table(name="tableName")** – Defines table name.

- @Id – Primary key.
 - @GeneratedValue – Auto key generation.
 - @Column(name="columnName") – Maps field to column.
 - @OneToMany, @ManyToOne – Relationship mapping.
- Annotations reduce the need for XML configuration.
-

✔ UNIT 4 – 5 MARKS QUESTIONS

1. Explain steps of Hibernate CRUD operations.

1. **Create Configuration object**

Loads hibernate.cfg.xml.

2. **Build SessionFactory**

One time creation.

3. **Open Session**

Represents DB connection.

4. **Begin Transaction**

Ensures atomic operations.

5. **Perform CRUD**

- save() → Insert
- get()/load() → Read
- update() → Update
- delete() → Delete

6. **Commit Transaction**

7. **Close Session and SessionFactory**

These steps provide easy database operations without SQL.

2. Explain Hibernate transactions.

Transactions in Hibernate manage the consistency of data.

Features:

- Ensures a group of operations act as a single unit.

- Provides **commit** (save changes) and **rollback** (undo changes).

Steps:

1. Open session
2. Begin transaction
3. Perform operations
4. Commit or rollback

Transactions prevent partial updates and maintain data integrity.

3. Explain generator classes and dialects.

Generator classes:

Used for generating primary keys automatically.

Examples:

- increment
- identity
- sequence
- uuid
- native

Dialects:

Specify database type so Hibernate generates correct SQL.

Examples:

- MySQLDialect
- OracleDialect
- PostgreSQLDialect
- SQLServerDialect

Dialects ensure compatibility with different databases.

UNIT 4 – 6 MARKS QUESTION (2+2+2)

(a) Explain Hibernate architecture.

Hibernate architecture has the following layers:

- **Application Layer** – Java application with DAO, service classes.
 - **Hibernate Layer** – SessionFactory, Session, Transaction, Query objects.
 - **Backend Layer** – Actual database.
- These components interact to perform ORM-based operations. Hibernate abstracts JDBC details and manages connection pooling and caching.
-

(b) Explain SessionFactory.

SessionFactory is a thread-safe, heavyweight object created once per application.

Functions:

- Reads configuration and mapping files.
 - Creates Session objects.
 - Maintains second-level cache.
 - Provides statistics and metadata.
- It should be created only once for performance.
-

(c) Explain HQL.

HQL (Hibernate Query Language) is an object-oriented query language:

- Uses class names instead of table names.
- Uses property names instead of column names.
- Supports joins, conditions, functions.

Example:

```
Query q = session.createQuery("from Student where id=1");
```

HQL is database-independent and easier than SQL.

UNIT 5 – 2 MARKS QUESTIONS

1. Define Spring Framework.

Spring is a lightweight and open-source framework used to develop enterprise-level Java applications using features like IOC, DI, MVC, and AOP.

2. What is IOC?

IOC (Inversion of Control) is a design principle where the control of object creation and management is transferred from the developer to the Spring container.

3. What is Dependency Injection?

Dependency Injection is the process of providing required objects (dependencies) to a class instead of creating them manually.

4. What is ApplicationContext?

ApplicationContext is the central container in Spring that manages beans, dependencies, configurations, and resources.

5. What is @Autowired?

@Autowired is used for automatic dependency injection by type.

6. Define Bean in Spring.

A bean is an object managed, created, and configured by the Spring IOC container.

UNIT 5 – 4 MARKS QUESTIONS

1. Explain Spring architecture.

Spring architecture is layered and includes:

1. **Core Container** – Beans, BeanFactory, ApplicationContext.
 2. **AOP Module** – Enables aspect-oriented programming.
 3. **Data Access Layer** – JDBC, ORM integration.
 4. **Web/MVC Layer** – Spring MVC framework.
 5. **Test Module** – Provides support for testing Spring applications.
- These layers work together to build flexible and maintainable applications.
-

2. Explain IOC and DI.

IOC (Inversion of Control):

- Shifts control of object creation to Spring container.
- Reduces coupling and increases modularity.

DI (Dependency Injection):

- Provides required dependencies to a bean.
- Achieved through:
 - Constructor Injection
 - Setter Injection
 - Field Injection

IOC is implemented using DI in Spring.

3. Explain autowiring types.

Spring supports several autowiring modes:

1. **By Type** – Injects bean based on class type.
 2. **By Name** – Matches bean name with property name.
 3. **Constructor** – Autowires using constructor parameters.
 4. **No** – No autowiring (default).
 5. **@Autowired** (Annotation-based) – Most widely used and easier.
-

4. Explain ApplicationContext types.

1. **ClassPathXmlApplicationContext** – Loads configuration from XML in classpath.
 2. **FileSystemXmlApplicationContext** – Loads XML from file system.
 3. **AnnotationConfigApplicationContext** – Loads configuration from Java-based annotations.
 4. **WebApplicationContext** – Used for web-based applications in Spring MVC.
-

✔ UNIT 5 – 5 MARKS QUESTIONS

1. Explain Spring MVC architecture.

Spring MVC is a model-view-controller based web framework.

Components:

1. **DispatcherServlet** – Front controller that receives all requests.
2. **Handler Mapping** – Finds the controller for the request.
3. **Controller** – Contains request-handling logic.
4. **Model** – Holds application data.
5. **View Resolver** – Identifies which view (JSP/HTML) to display.
6. **View** – Final output shown to user.

Flow: Request → DispatcherServlet → Controller → Service → Model → ViewResolver → View.

2. Explain annotation-based configuration.

Spring supports Java-based configuration using annotations:

- **@Component** – Marks class as bean.
 - **@Autowired** – Injects dependencies automatically.
 - **@Configuration** – Declares configuration class.
 - **@Bean** – Declares a bean manually inside class.
 - **@Controller, @Service, @Repository** – Stereotype annotations.
This removes the need for XML, making code cleaner and faster to develop.
-

3. Explain Dependency Injection in detail.

DI is the core of Spring IOC.

Types:

1. **Constructor Injection** – Dependencies provided through constructor.
2. **Setter Injection** – Uses setter methods.
3. **Field Injection** – Uses **@Autowired** on fields.

Benefits:

- Loosely coupled code
 - Easy testing
 - Reusable and maintainable components
- Spring container reads configuration and injects dependencies automatically.
-

✔ UNIT 5 – 6 MARKS QUESTION (2+2+2)

(a) Explain Spring IOC.

IOC is a design principle where the control of object creation and binding is transferred from the program to the Spring container.

The container creates objects, configures them, and manages their lifecycle.

IOC removes tight coupling and improves scalability.

(b) Explain DI.

Dependency Injection provides external objects to classes instead of creating them inside.

DI types:

- Constructor Injection
- Setter Injection
- Field Injection (@Autowired)

Advantages:

- Loose coupling
- Easy to replace modules
- Better unit testing

DI is the implementation method of IOC.

(c) Explain Spring MVC dispatcher flow.

1. Client sends request to **DispatcherServlet**.
2. DispatcherServlet looks up **Handler Mapping**.

3. The request is forwarded to the correct **Controller**.
4. Controller processes data and returns **Model & View name**.
5. **ViewResolver** identifies correct JSP/HTML file.
6. DispatcherServlet renders view and sends response to client.

This flow clearly separates business logic, UI, and request handling.

UNIT 6 – 2 MARKS QUESTIONS

1. What is Spring Boot?

Spring Boot is a framework built on top of Spring that simplifies application development using auto-configuration and embedded servers.

2. What are starters?

Starter are pre-configured dependency packages in Spring Boot that reduce manual dependency management. Example: spring-boot-starter-web.

3. What is auto-configuration?

Auto-configuration automatically configures required beans based on the classpath and project settings.

4. What is actuator?

Actuator provides production-ready features like health checks, metrics, monitoring, and application info.

5. What is Spring Boot CLI?

Spring Boot CLI is a command-line tool used to quickly create and run Spring applications using Groovy scripts.

6. Define REST API.

REST API is an architectural style that uses HTTP methods (GET, POST, PUT, DELETE) to perform operations on resources.

✔ UNIT 6 – 4 MARKS QUESTIONS

1. Explain features of Spring Boot.

Major features include:

1. **Auto-configuration** – Reduces manual setup.
2. **Starter dependencies** – Pre-configured dependencies.
3. **Embedded servers** – Tomcat, Jetty built-in.
4. **Actuator support** – Monitoring and metrics.
5. **Spring Boot CLI** – Faster development.
6. **Production-ready** – Logging, error handling, runtime configuration.

2. Explain auto-configuration.

Auto-configuration in Spring Boot automatically creates and configures Spring beans based on project dependencies.

- Removes need for XML or manual Java config.
- Uses `@EnableAutoConfiguration` and internal conditions like:
 - `@ConditionalOnClass`
 - `@ConditionalOnMissingBean`It saves development time and reduces boilerplate code.

3. Explain starter dependencies.

Starter dependencies combine commonly used libraries into simple packages.

Examples:

- `spring-boot-starter-web` – MVC + Tomcat + JSON.
 - `spring-boot-starter-data-jpa` – JPA + Hibernate.
 - `spring-boot-starter-security` – Security modules.
- Starters help in quick and clean project setup by reducing individual dependency configuration.

✔ UNIT 6 – 5 MARKS QUESTIONS

1. Explain CRUD using Spring Boot + JPA.

Spring Boot makes CRUD easy with JPA:

1. **Entity Class** – Mark with `@Entity`.
2. **Repository Interface** – Extend `JpaRepository`.
3. **Service Layer** – Contains business logic.
4. **Controller Layer** – Exposes CRUD operations through REST.
5. **Database Configuration** – Provided in `application.properties`.

Methods:

- `save()` → Create/Update
- `findAll()` / `findById()` → Read
- `deleteById()` → Delete

Spring Boot auto-configures JPA, making CRUD very simple.

2. Explain Spring Boot REST architecture.

Spring Boot REST architecture includes:

1. **Controller Layer** – Handles HTTP requests using `@RestController`.
2. **Service Layer** – Business logic.
3. **Repository Layer** – Database operations using JPA.
4. **Models/Entities** – Data structure mapped to DB.
5. **Response Handling** – JSON responses automatically provided by Jackson.

REST uses HTTP verbs: GET, POST, PUT, DELETE for resource manipulation.

3. Explain Spring Boot Profiles.

Profiles allow separating configuration for different environments like dev, test, and production.

Use annotation:

```
@Profile("dev")
```

Configurations stored as:

- `application-dev.properties`
- `application-prod.properties`

Features:

- Enables environment-specific settings
 - Prevents production errors
 - Helps in better deployment workflow
-

(a) Explain Spring Boot architecture.

Spring Boot architecture consists of:

1. **Auto-configuration Layer** – Automatically configures the application.
2. **Starter Dependencies** – Simplified dependency management.
3. **Spring Container (IOC)** – Creates and manages beans.
4. **Embedded Servers** – Tomcat, Jetty for running applications without external setup.
5. **Application Layer** – Controllers, Services, Repositories.

This architecture leads to faster development and easy deployment.

(b) Explain auto-configuration.

Auto-configuration inspects the classpath and automatically configures the required beans.

Example: If Spring Boot finds spring-web on the classpath, it configures MVC automatically.

Uses:

- `@EnableAutoConfiguration`
 - Conditional annotations
- Reduces manual configuration and speeds up development.
-

(c) Explain database integration.

Spring Boot simplifies database integration using Spring Data JPA.

Steps:

1. Add dependency: spring-boot-starter-data-jpa.
2. Configure database in application.properties.
3. Create entity class with `@Entity`.
4. Create repository interface using `JpaRepository`.
5. Perform CRUD operations through service and controller.

Spring Boot also configures DataSource, Hibernate, and TransactionManager automatically.